Introduction

This project asked me to build a systematic trading strategy for BAC stock under realistic constraints: all-in or all-out trades, wash-sale restrictions, and different tax rates for short-term and long-term gains. I decided to lean on a Generative AI assistant throughout the process. What follows is a reflection on how GenAI shaped my workflow, what it did well, where it fell short, and what I could have done to use it more effectively.

How GenAI Influenced My Process

1. Bootstrapping ideas and structure

At the outset I needed a rules-based approach that was defensible. GenAI immediately offered a moving average crossover strategy. That suggestion was not unique, yet it gave me a concrete starting point that met the requirement for an ex-ante indicator. It also provided skeletal Python code to ingest the CSV, compute indicators, and simulate trades. That saved me time on boilerplate and let me focus on gaps such as taxes and wash-sale logic.

2. Rapid prototyping and iteration

When the first code sample failed due to a length mismatch in a portfolio value vector, GenAI quickly located and fixed the issue. More importantly, once I realized the initial solution ignored accurate tax flows and simplified the wash-sale rule, I could ask the model to redesign the simulation. It produced a more explicit state machine approach, suggested logging each trade with indicator values, and guided me to include after-tax cash updates. Each iteration was faster because I could say, “apply capital gains tax at the moment of sale” or “block buys for 30 days after any loss” and get updated code in minutes.

3. Scaling to optimization

After we had a working simulator, I asked the model to go beyond a single set of rules. GenAI proposed and then executed a grid search across thresholds for buy and sell bands and RSI levels. When I asked for Bayesian optimization to find better parameters, it produced a full loop with a Gaussian Process, Expected Improvement, and Latin Hypercube initialization. I would have needed a long time to write and debug that from scratch; with GenAI it took a few prompt exchanges.

4. Documentation and deliverables

The assignment required a technical appendix in both PDF and Excel formats. GenAI wrote the code to export tables, metrics, and charts, then generated a formatted PDF. When the first PDF layout looked messy, I requested a cleaner table and it rebuilt the document with a landscape format and column widths. GenAI essentially became a formatting assistant on demand.

Strengths of GenAI in This Context

Speed and breadth of ideation
GenAI gave me several pathways quickly: crossover strategies, mean reversion with SMA bands, RSI thresholds, and regime filters. It also knew to combine a long-term trend filter with short-term reversal signals. The tool does not tire, so I could ask for more variants without worrying about time.

Code scaffolding and debugging support
GenAI reliably produced runnable Python code, including data cleaning, indicator computation, and plotting. When errors popped up, it proposed plausible fixes. It also helped me modularize logic.

Ability to operationalize constraints
Wash-sale and tax rules are easy to describe yet annoying to code. GenAI could translate those legalistic constraints into explicit logic: track the date of each loss, mark a future buy blackout, compute holding periods, classify gain types, and deduct tax immediately. The first try was incomplete, but it understood the concept, which made refinement easier.

Formatting and reporting
Producing an Excel file with multiple sheets and a neat PDF is tedious but important for a technical appendix. GenAI wrote the export code, handled numeric formatting, and re-rendered the PDF when I complained about line breaks. This freed me to think about content quality rather than layout quirks.

Statistical tooling on demand
The model pulled in Gaussian Process regression, Expected Improvement, and Latin Hypercube sampling without me having to look up syntax. It also returned risk metrics like max drawdown and Sharpe ratio on request. That level of statistical hygiene would normally require more time or a prebuilt library.

Limitations of GenAI in This Context

Initial superficiality on domain rules
The first solution glossed over several essential constraints. The model said it handled taxes and wash-sale, but the code only partially did. Gains were taxed conceptually but not subtracted from cash. Wash-sale logic blocked buying once, not for every loss. If I had accepted the first pass, I would have overestimated final performance by a wide margin.

Inconsistent rigor without prompting
GenAI tends to produce something that looks correct, but without explicit instructions it will skip edge cases. For example, it needed reminders to prevent look-ahead bias, to choose whether to trade at close or next day's open, and to store indicator values for each trade to satisfy "metrics used to make the decision."

Optimization without validation
The model can optimize parameters on the entire data sample, but it will not automatically recommend walk-forward validation or out-of-sample testing unless I ask. That is a classic overfitting trap. The tool optimizes exactly what I ask for. If I do not insist on robust validation, it will happily give me a curve-fit solution.

Possible numerical bugs and silence on warnings
During Bayesian optimization, scikit-learn emitted kernel bound warnings. GenAI ignored them unless I asked about stability. It also used arbitrary hyperparameters for the GP kernel. A human might immediately question the stationarity assumption or consider a different acquisition function.

Formatting brittleness
PDF generation needed several tweaks. Multi-cell text wrapped awkwardly, numeric columns needed width adjustments, and fonts were inconsistent. GenAI can fix these issues, but only after I notice and specify what is wrong.

No understanding of real brokerage friction
The simulation ignored slippage, commissions, and bid-ask spreads. That is acceptable for a class project, but the model did not surface these caveats proactively. It also did not suggest a risk-free rate for Sharpe or account for idle cash returns unless prompted.

Self Assessment: How I Could Have Been a More Effective GenAI User

1. Provide a tighter specification up front
If I had written a mini spec at the beginning, the first code draft would have been closer to correct. For example:

- "Deduct tax from cash immediately after each sale."
- "Track each trade's lot for holding period."
- "Implement wash-sale for every loss sale with a list of restricted intervals."
- "Trade on next day's open to avoid same-day look-ahead."
  Giving these rules explicitly would have reduced rework.

2. Define acceptance tests before coding
I could have said: "After any loss sale on date D, assert that no BUY trades occur between D+1 and D+30." or "After a profitable sale, tax should reduce cash by exactly 15 percent or 25 percent of the gain." This kind of test-driven prompt would force GenAI to embed checks and raise flags.

3. Ask for multiple strategies from the start
I initially let the model settle on a single moving average crossover before expanding. If I had asked, "Generate three fundamentally different rule families and outline pros and cons for each under taxes and wash-sale constraints" I could have explored a broader landscape earlier.

4. Enforce out-of-sample validation
I should have required a rolling or walk-forward test from the moment optimization was introduced. The instruction would be: optimize on years 1 to 3, test on year 4, roll forward by one year, then average. Without that, I risk overfitting the five year sample.

5. Request clarity on execution timing and data availability
Telling the model, "Assume you only know today's closing price after the market closes, so trades execute at next day's open" would help eliminate subtle bias. I also could have asked for a sensitivity analysis: "What happens if execution price slips by 0.1 percent?"

6. Use shorter, more targeted prompts
Some of my prompts were long and mixed requirements with narrative. GenAI performs better when a prompt is a checklist. Segmenting my requests into small tasks (wash-sale logic, tax engine test, final report formatting) might have reduced errors.

7. Keep a human-in-the-loop skepticism
The model's confidence can be misleading. I should habitually verify numbers, recompute simple metrics by hand on a few trades, and cross-check tax math. Trust but verify.

8. Leverage GenAI's ability to critique itself
I could have asked, "List five possible mistakes in the current implementation" or "Explain how this code could fail during extreme volatility." This pushes the model to adopt an adversarial reviewer mindset.

9. Plan the final deliverables early
I waited until late to request the technical appendix. If I had described the final PDF and Excel layout at the start, GenAI could have stored metrics and formatted data in a way that dropped neatly into the reports.

Conclusion

GenAI functioned like a fast but somewhat careless junior analyst. It sketched viable strategies, wrote and debugged code, and generated polished outputs on command. The flipside is that it lacked the discipline to enforce every constraint or catch every subtle bug unless I explicitly instructed it to do so. The experience highlighted that the value of GenAI is not just in faster coding or drafting. The real advantage comes when I steer it with precise requirements, set up verification steps, and ask it to critique its own work.

Looking back, I could have been a more effective GenAI user by front-loading my constraints, using test-driven prompts, requiring validation protocols, and challenging the model to find its own weaknesses. When those practices are in place, GenAI becomes a powerful partner rather than a risky shortcut. With that mindset, I can harness its speed and breadth while keeping the rigor and accountability firmly in my hands.